KEMENTERIAN PENDIDIKAN TINGGI
JABATAN PENDIDIKAN POLITEKNIK DAN KOLEJ KOMUNITI

POLITEKNIK MALAYSIA

JABATAN TEKNOLOGI MAKLUMAT DAN KOMUNIKASI

MINI PROJECT 1
SESI I : 2025/2026

DFP40443 FULL STACK WEB DEVELOPMENT

TARIKH : MINGGU KE-6
MASA : 0 JAM (F2F) + 4 JAM (NF2F)

Kertas soalan ini mengandungi **enam (6)** halaman bercetak.

POLITEKNIK MALAYSIA
SEBERANG PERAI

DFP40443 FULL STACK WEB DEVELOPMENT

SESI 1 2026/2027

NO.	REGISTRATION NO	NAME
1	10DIT24F1167	SHIVAASHINEE MUNUSAMY
2	10DIT23F2003	DIVYA THANABALASINGAM
3		
4		

ASSESSMENT : MINI PROJECT 1

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY | MINI PROJECT 1 SESSION: 2 2025/2026

COURSE CODE : DFP40443	**COURSE NAME :** FULL STACK WEB DEVELOPMENT
Duration : 4 Hours(NF2F)	**Total Marks _____/ 100**
CLO 1 : Integrate PHP and MySQL with Bootstrap to build database-driven web applications projects (P4, PLO3)	

INSTRUCTIONS:

1. This assessment consists of TWO (2) main practical tasks designed to evaluate your proficiency in building database-driven web applications using PHP, MySQL, and the MVC architecture.
2. You are required to submit your work in softcopy form, consisting of:
    a. **Source Code**: A compressed folder (.zip) containing all project files (Model, View, Controller folders, CSS, and JS).
    b. **Database Export**: A .sql file of your database schema.
    c. **Project Documentation**: A technical report in PDF format.
3. Read the questions carefully and refer to the attached **RUBRIC** for the marking scheme.
4. **PLAGIARISM WARNING**: Marks will be strictly deducted, or a grade of zero may be awarded, if a student is found to be copying or sharing code from/to others. All logic and implementation must be your own work.

Scenario:

As a Full Stack Web Developer, you are required to develop a Web Based PSP Student Portal. For the front-end interface, you are encouraged to utilize the Bootstrap framework via CDN to design a responsive and user-friendly layout. The system should demonstrate the implementation of:

1. MVC Architecture and modular coding
2. Session management and Secure Coding Practices
3. MySQL Database Integration and Database Relationships
4. CRUD operations

PART 1: PROFILE MANAGEMENT & ACCOUNT SECURITY

In this section, you must ensure the system can handle students' personal information securely based on an active session

1.1 Student Profile Page

- **Design:** Create a profile page that displays the student's **Name, NRIC,** and **Program**.
- **Session Management:** The information displayed must be based strictly on the currently logged-in student using the $_SESSION variable.
- **Access Control:** The system must ensure that students cannot access or view the profile information of other students

1.2 Password Exchange Module

- **Functionality:** Build a password update feature containing input fields for: **Old Password, New Password,** and **Confirm Password**.
- **Validation Logic:** The system must verify the validity of the **Old Password** against the record in the database before allowing any changes.
- **Data Security:** Students are required to use **Password Hashing** techniques to process the new password before it is stored in the database.

PART 2: GRADE MANAGEMENT SYSTEM REVAMP

You are assigned to perform a revamp so that all student data is managed entirely through a **MySQL database**.

2.1 Dynamic database (MySQL)

• implement database access using **SQL queries** to retrieve student records.

2.2 Implementation of CRUD Operations

You are required to build the interface and logic for the following operations:

Operation	Technical Requirements
CREATE	Create a form to add new student records (**Name, IC, and Marks**) directly into the database.
READ	Display a complete list of students and their marks from the database in a table format.
UPDATE	Provide functionality to update existing student information or marks.
DELETE	Provide functionality to remove student records. **Note:** Ensure there is a **confirmation** prompt before data is permanently deleted.

TECHNICAL SPECIFICATIONS

- **Architecture:** The code must strictly follow the **MVC** pattern (Model, View, Controller).
- **Modular Coding:** Use modular techniques such as include or require for reusable components like database connections.
- **Security:** Implement **Prepared Statements** for all SQL operations to prevent SQL Injection attacks.
- **Validation:** Include both client-side and server-side validation for all forms.

INTRODUCTION

The Web Based PSP Student Portal is a database-driven web application developed for managing student information and academic marks. The system was developed using PHP, MySQL and Bootstrap.

The system provides two main functions. The first function is Profile Management and Account Security, where logged-in students can view their own profile information and change their password. The second function is Grade Management, where student records and marks can be added, viewed, updated and deleted.

The system applies the Model-View-Controller (MVC) architecture to separate the application into different components. MySQL is used to store the system data, while prepared statements are used for database operations to improve SQL security.

OBJECTIVE

The objectives of this project are:

1. To develop a Web Based PSP Student Portal using PHP, MySQL and Bootstrap.
2. To implement a secure login system using session management.
3. To allow logged-in students to view their own profile information and change their password securely.
4. To implement Create, Read, Update and Delete (CRUD) operations for student grade records.
5. To apply MVC architecture, modular coding, prepared statements and form validation in the development of the system.

DATABASE DESIGN / ERD

Database is: psp_student_portal

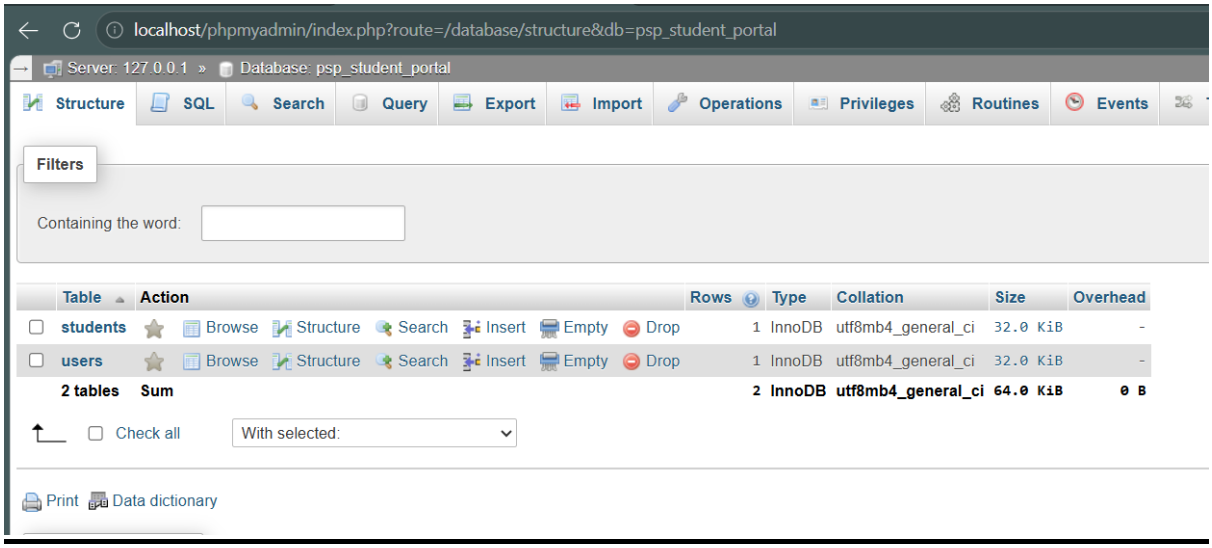


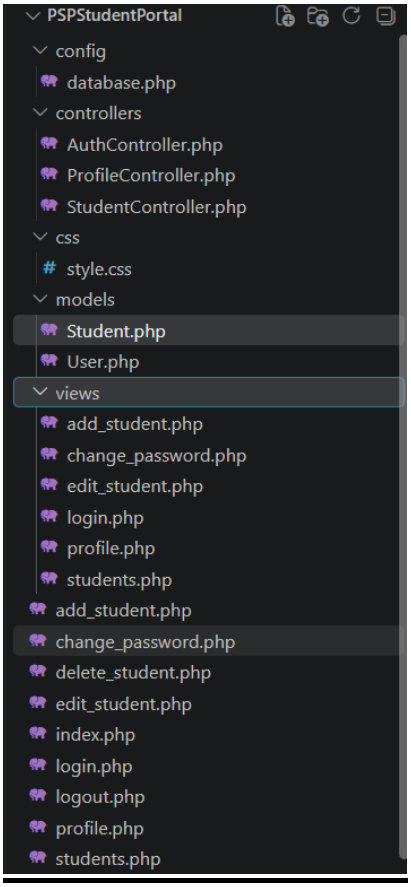
Figure 1: Entity Relationship Diagram of PSP Student Portal

USERS
PK id
name
nric
program
password

+

STUDENTS
PK id
name
ic
marks

MVC Project Folder Structure



MODEL
The Model handles database-related operations. User.php manages user information, login authentication and password updates, while Student.php manages student records and CRUD operations.

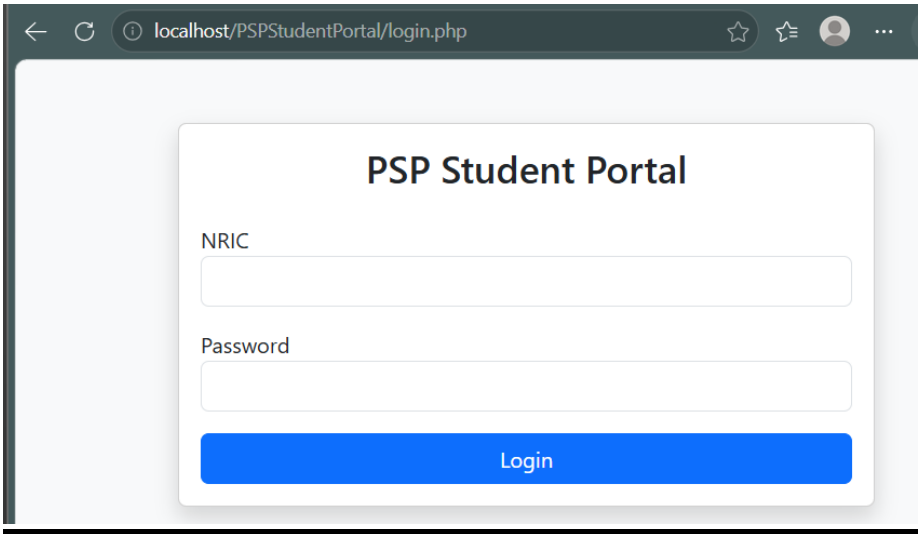
VIEW
The View is responsible for displaying the user interface, including login, profile, password change and student grade pages.

CONTROLLER
The Controller receives requests from the application and connects the View with the appropriate Model. AuthController.php manages authentication, ProfileController.php manages profile-related functions and StudentController.php manages student records.

DATABASE
database.php provides a reusable database connection using the Database class.

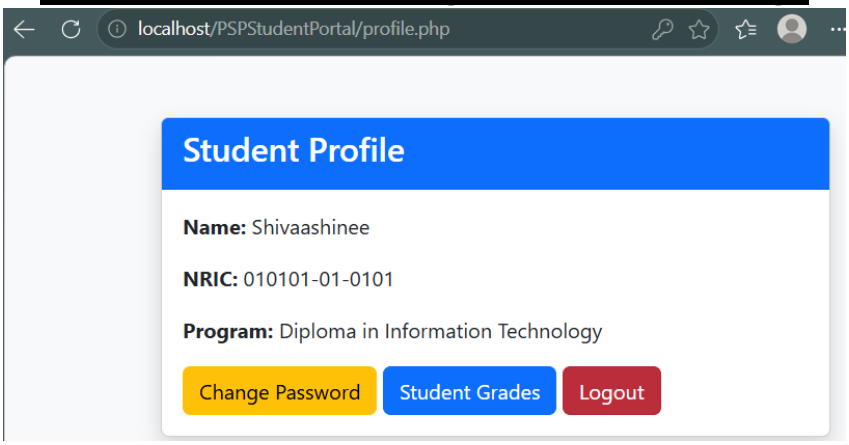
Student Login Page

NRIC: 010101-01-0101
PASSWORD: 123456789



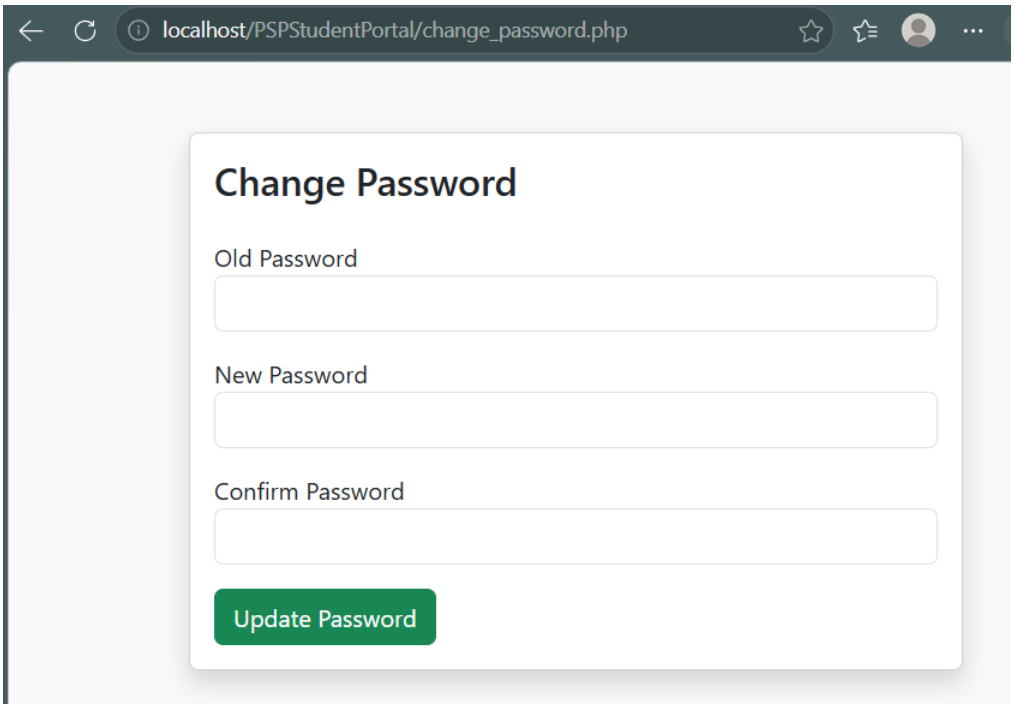
The login page allows students to enter their NRIC and password. The system verifies the entered credentials against the users table. When authentication is successful, the user's ID is stored in the PHP session.

Successful Student Login and Profile Page

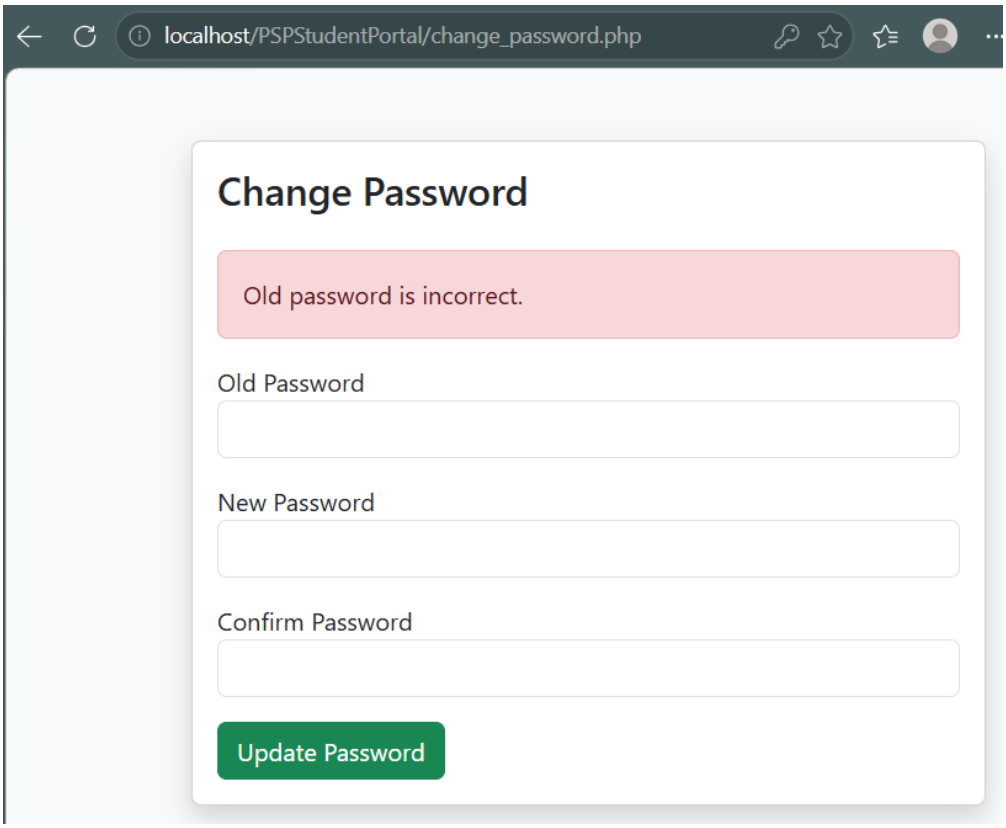


The assignment requires the profile to display these details based on the **currently logged-in student using $_SESSION**, and students must not be able to view another student's profile.

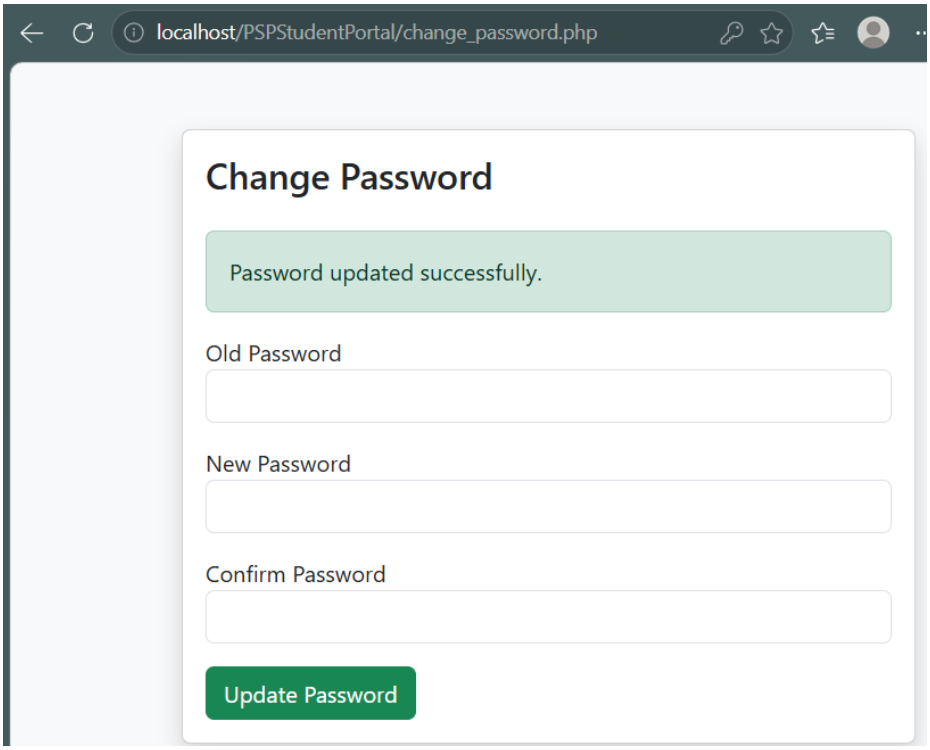
Password Security



The old password is checked before the new password is saved, and the new password is hashed before storage.

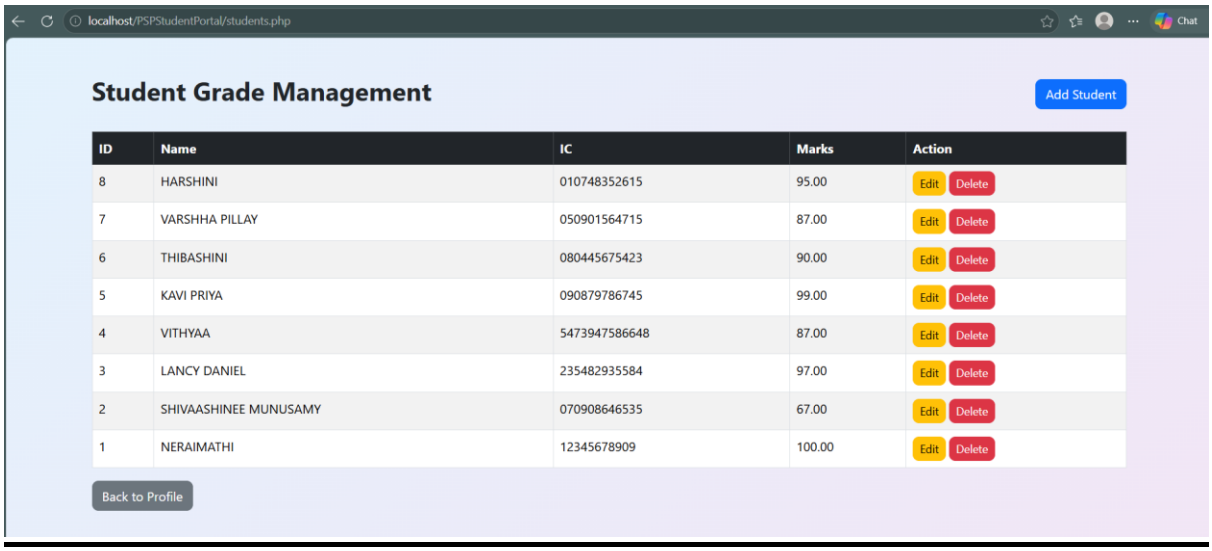


Enter an incorrect old password.



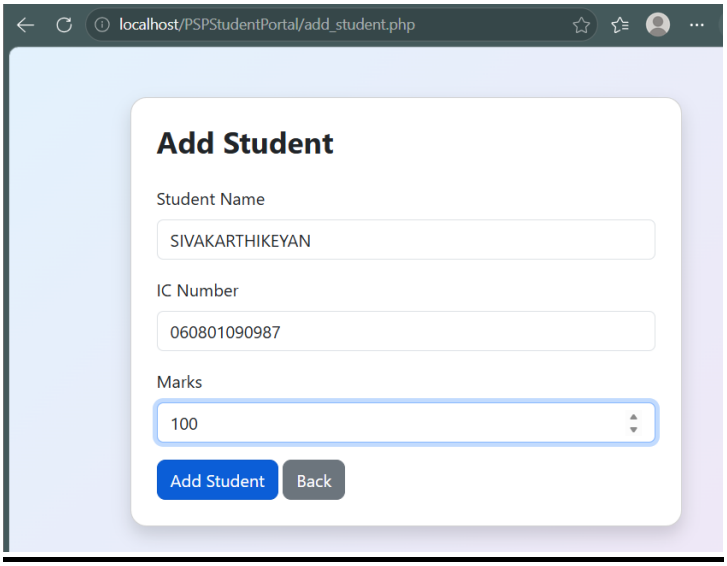
Show the successful password-change message.

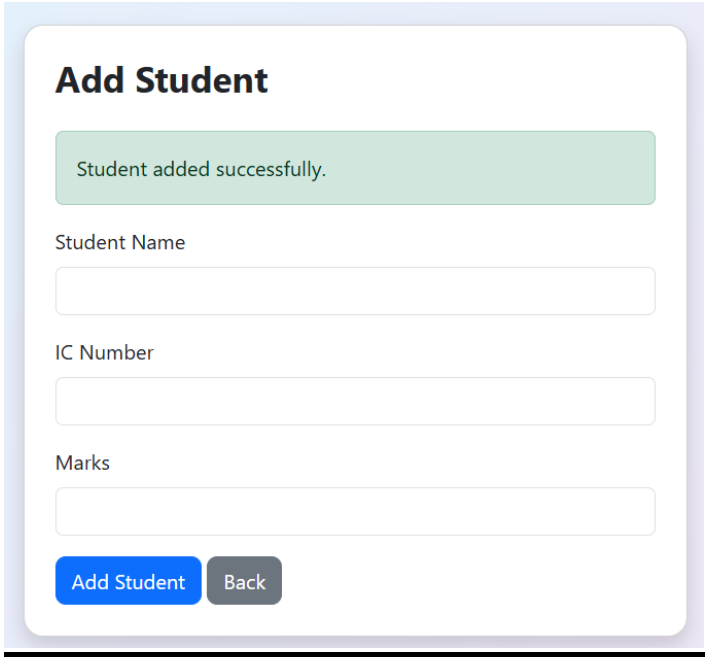
Grade Management System



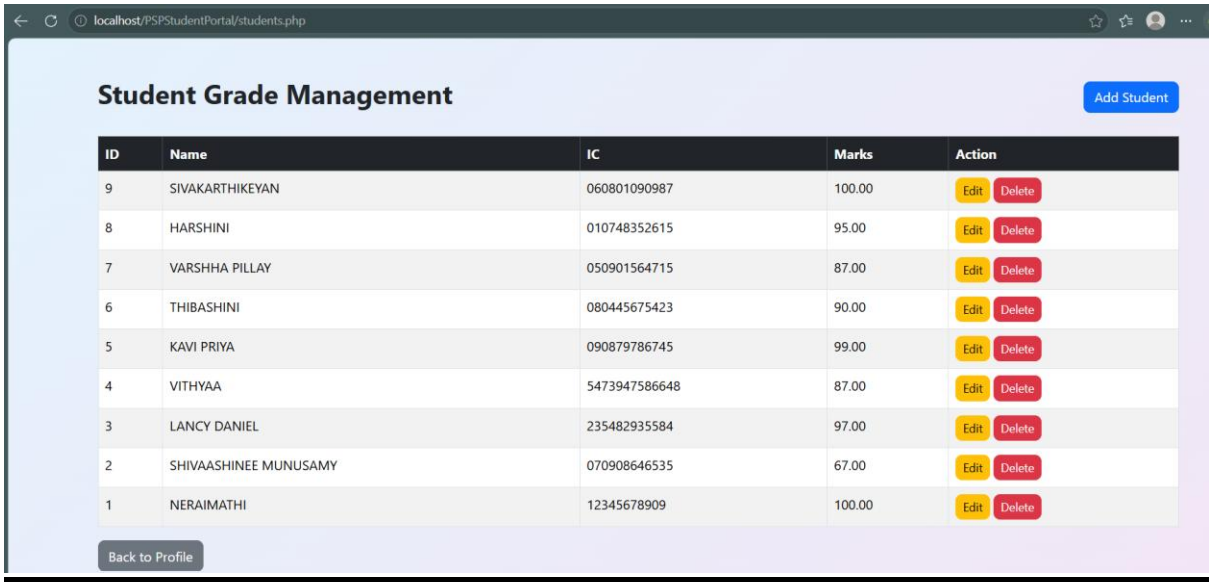
The Grade Management System stores student information and marks in the MySQL database. The system supports Create, Read, Update and Delete operations

Add Student (CREATE)

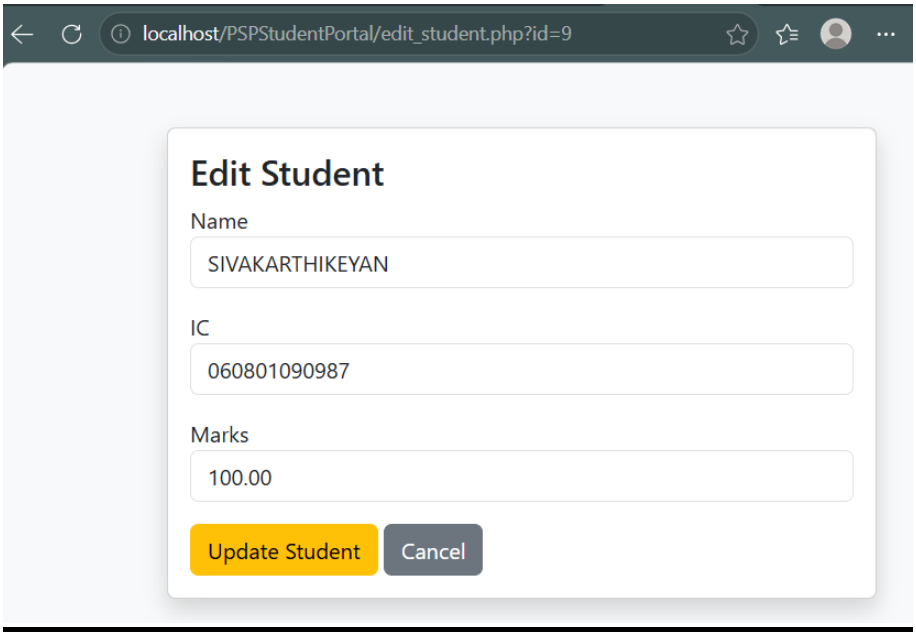


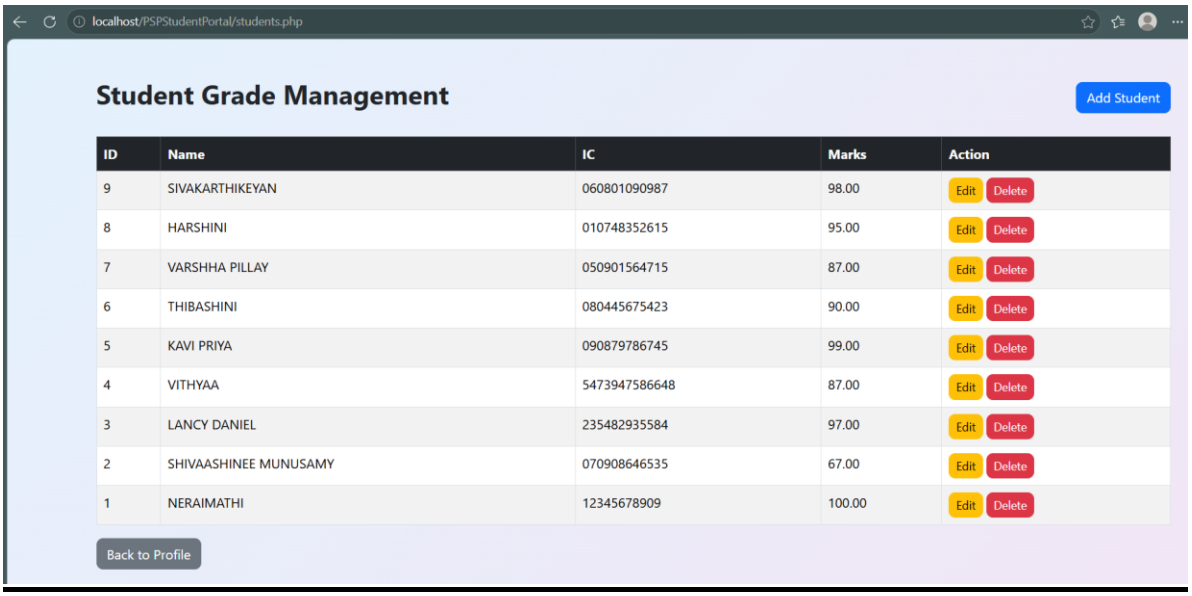


Student Records Retrieved from MySQL Database (READ)



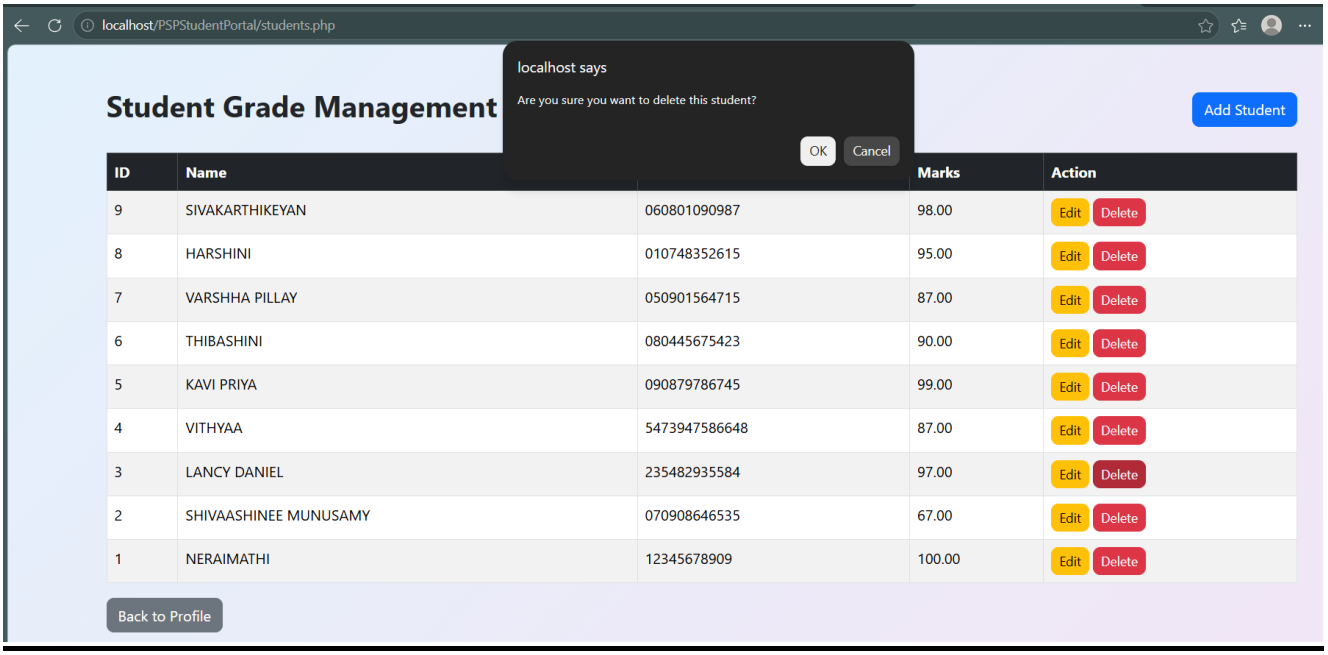
Edit (UPDATE)

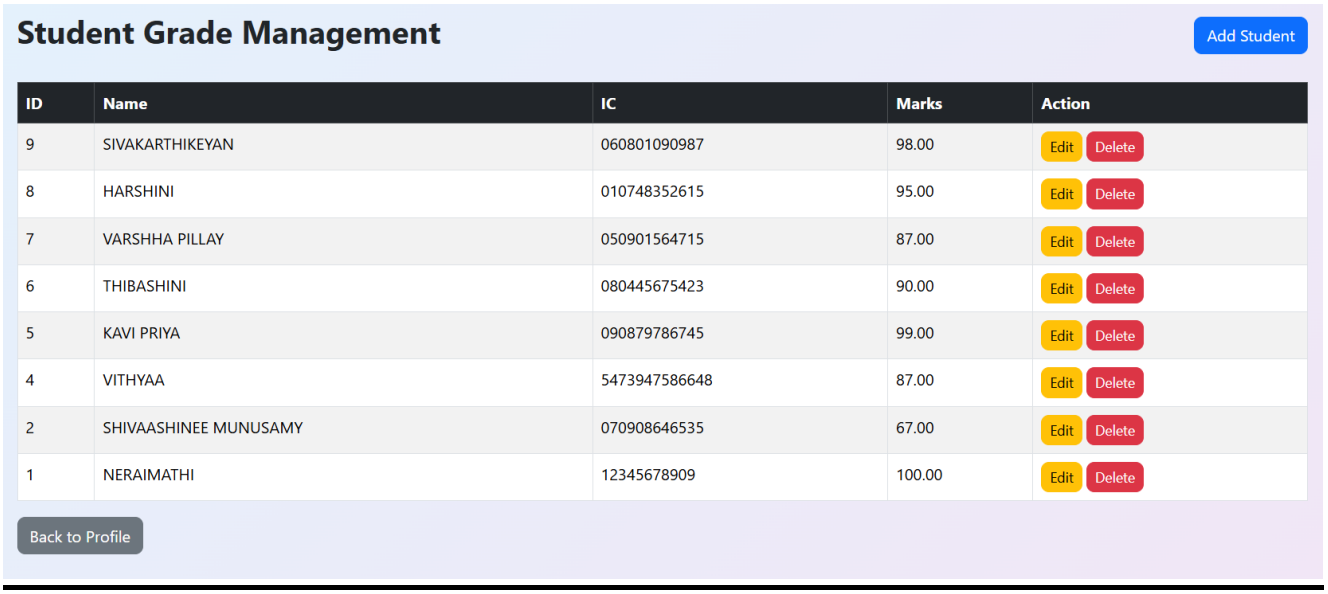




Show the updated student with 98.

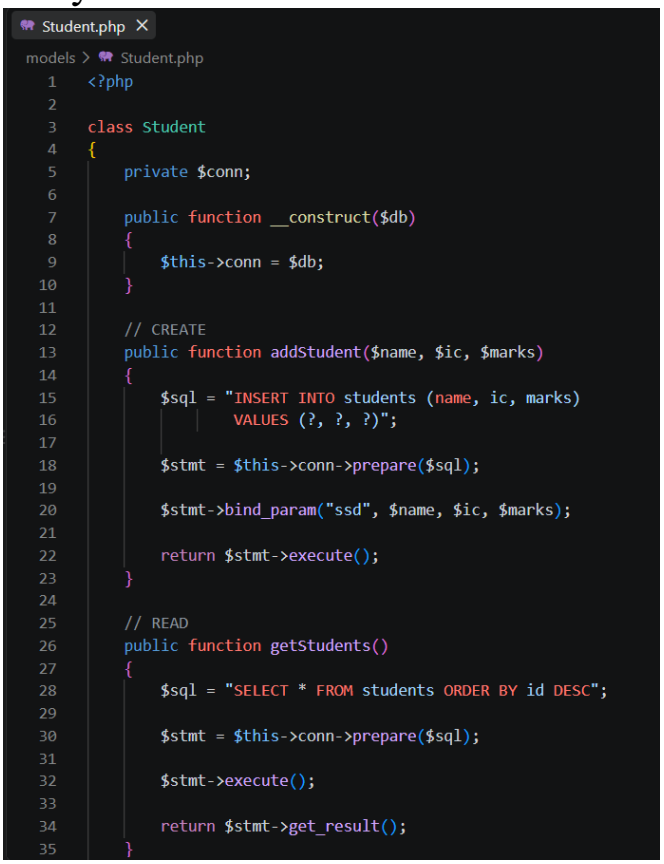
DELETE


localhost/PSPStudentPortal/students.php
localhost says
Are you sure you want to delete this student?
OK
Cancel
Student Grade Management
Add Student
ID
Name
Marks
Action
9 SIVAKARTHIKEYAN 060801090987 98.00 Edit Delete
8 HARSHINI 010748352615 95.00 Edit Delete
7 VARSHHA PILLAY 050901564715 87.00 Edit Delete
6 THIBASHINI 080445675423 90.00 Edit Delete
5 KAVI PRIYA 090879786745 99.00 Edit Delete
4 VITHYAA 5473947586648 87.00 Edit Delete
3 LANCY DANIEL 235482935584 97.00 Edit Delete
2 SHIVAASHINEE MUNUSAMY 070908646535 67.00 Edit Delete
1 NERAIMATHI 12345678909 100.00 Edit Delete
Back to Profile


Student Grade Management
Add Student
ID
Name
IC
Marks
Action
9 SIVAKARTHIKEYAN 060801090987 98.00 Edit Delete
8 HARSHINI 010748352615 95.00 Edit Delete
7 VARSHHA PILLAY 050901564715 87.00 Edit Delete
6 THIBASHINI 080445675423 90.00 Edit Delete
5 KAVI PRIYA 090879786745 99.00 Edit Delete
4 VITHYAA 5473947586648 87.00 Edit Delete
2 SHIVAASHINEE MUNUSAMY 070908646535 67.00 Edit Delete
1 NERAIMATHI 12345678909 100.00 Edit Delete
Back to Profile

SQL Security

Prepared statements are used for database operations in the PSP Student Portal. Instead of directly placing user input into SQL statements, placeholders are used and the values are bound separately using bind_param().
For example, the Student model uses a prepared statement when inserting student information:
INSERT INTO students (name, ic, marks) VALUES (?, ?, ?)
The values are then passed using bind_param(). This helps protect the application against SQL injection attacks.Prepared statements are consistently used for the database operations in the system.



```php
    // READ ONE
    public function getStudent($id)
    {
        $sql = "SELECT * FROM students WHERE id = ?";

        $stmt = $this->conn->prepare($sql);

        $stmt->bind_param("i", $id);

        $stmt->execute();

        return $stmt->get_result()->fetch_assoc();
    }

    // UPDATE
    public function updateStudent($id, $name, $ic, $marks)
    {
        $sql = "UPDATE students
                SET name = ?, ic = ?, marks = ?
                WHERE id = ?";

        $stmt = $this->conn->prepare($sql);

        $stmt->bind_param("ssdi", $name, $ic, $marks, $id);

        return $stmt->execute();
    }

    // DELETE
    public function deleteStudent($id)
    {
        $sql = "DELETE FROM students WHERE id = ?";

        $stmt = $this->conn->prepare($sql);
```

```php
    }

    // DELETE
    public function deleteStudent($id)
    {
        $sql = "DELETE FROM students WHERE id = ?";

        $stmt = $this->conn->prepare($sql);

        $stmt->bind_param("i", $id);

        return $stmt->execute();
    }
}

?>
```

INFORMATION AND COMMUNICATION TECHNOLOGY DEPARTMENT
MINI PROJECT 1 RUBRIC | **SESI 1 2026/2027**

COURSE CODE	**DFP40443**		**COURSE NAME**	**Full Stack Web Development**			
ASSESSMENT TASK	**Mini Project 1**		**LECTURER'S NAME**				
STUDENT'S NAME	**1**		**REGISTRATION NUMBER**	**1**			
	2			**2**			
	3			**3**			
	4			4			
Category / Score	**4 (Excellent)**	**3 (Good)**	**2 (Fair)**	**1 (Poor)**	**Weight (%)**	**Student's Score (out of 4)**	**Marks (Score × Weight / 4)**
MVC & Modular Structure	Clear MVC separation; highly modular code using include/require.	MVC used but with minor organization issues.	Limited modularity; logic and view are not fully separated.	No MVC structure; unorganized code.	**10**		0
Session & Profile (Q1)	Profile functions perfectly via $_SESSION; strict access control.	Session works, but minor flaws in access control logic.	Profile displays data but session management is unstable.	Failed to display information based on active sessions.	**15**		0
Password Security (Q1)	Accurate old password validation and perfect hashing implementation.	Hashing is used but validation messages are unclear.	Update works but lacks hashing or old password validation.	Passwords stored in plain text; no security measures.	**15**		0
Database & CRUD (Q2)	All CRUD operations work perfectly; seamless migration from array to SQL.	CRUD works but with minor bugs in one function (e.g., Update).	CRUD is partially complete with several logic errors.	Failed to connect to MySQL; CRUD is non-functional.	**25**		0
SQL Security	Consistently uses Prepared Statements for	Uses Prepared Statements for most	Queries work but are vulnerable to SQL Injection (raw variables).	No security considerations in SQL code.	**10**		0

	all database interactions.	queries with minor omissions.					
Technical Documentation	Complete report: ERD, MVC structure, functional screenshots, and deep technical explanations.	Complete report but lacking detailed technical descriptions	Report provided but disorganized or missing key screenshots.	Very brief documentation; missing format or evidence of testing.	**25**		0
						Total score	**0**